Projeto do Semestre — Campus Acessível

Sistema de Inventário e Priorização de Intervenções de Acessibilidade

UNASP — Centro Universitário Adventista de São Paulo · Campus Engenheiro Coelho
Disciplina: Construção de Algoritmos e Programação (G0719.2) — 2026/2
Cursos: Engenharia de Computação (GBECOM23A) e Sistemas de Informação (GBSINF23A)
Professor: Eduardo Mendes Oliveira
Peso: 20% da média final (PJ) · nota de 0,0 a 10,0

1. O que este projeto é

A coordenação de curso precisa de duas coisas que hoje não existem: um **inventário confiável** das barreiras de acessibilidade do campus e uma **recomendação defensável** de quais intervenções fazer primeiro, dado que o orçamento nunca cobre todas.

Seu grupo é contratado como equipe técnica para resolver isso em um setor do campus. O trabalho tem três partes inseparáveis: ir a campo e medir, construir um programa que processe o que foi medido, e **sustentar a recomendação que sai do programa** diante de quem vai gastar o dinheiro.

O produto deste projeto não é o programa. O produto é a recomendação, e o programa é o instrumento que a torna verificável. Quem entende a diferença entre as duas coisas tem o projeto praticamente resolvido; quem não entende vai entregar um código que roda e não vai conseguir dizer por que a resposta dele é a resposta certa.

2. O problema, em uma frase

> Dado o conjunto de barreiras de acessibilidade que o grupo mediu em seu setor, e um teto de orçamento, **quais intervenções devem ser executadas e em que ordem** — e por quê essa é a melhor escolha possível dentro do teto?

Não há gabarito. Setores diferentes produzem respostas diferentes, e dois grupos com dados parecidos podem chegar legitimamente a recomendações distintas se tiverem adotado critérios distintos. O que não é aceitável é uma recomendação que o grupo não consiga explicar.

3. O que o grupo entrega

	Entregável	Formato
1	**Inventário do setor**, coletado em campo pelo grupo	ficha de campo preenchida + arquivo do sistema
2	**Sistema Campus Acessível**, em Python, executado no terminal	código-fonte em módulos
3	**Relatório técnico de 4 a 6 páginas** com os cinco achados (seção 9)	PDF
4	**Caderno de bordo** do grupo	PDF ou arquivo de texto
5	**Pitch de 5 minutos** com demonstração ao vivo e arguição	apresentação

4. Composição dos grupos

Curso	Turma	Integrantes por grupo	Grupos previstos
Engenharia de Computação	GBECOM23A	**4**	13
Sistemas de Informação	GBSINF23A	**3**	8

Os grupos são definidos na aula de lançamento — **16/09 (quarta)** em Engenharia de Computação e **17/09 (quinta)** em Sistemas de Informação. **Cada grupo recebe um setor diferente do campus**, sorteado na mesma aula. Não existem dois grupos da mesma turma no mesmo setor: o seu inventário é único e não há nada que se possa copiar de outro grupo.

4.1 As duas fases do projeto

O projeto corre em duas fases, com propósitos distintos.

Fase 1 — Levantamento (16/09 a 15/10). O grupo vai a campo e produz o inventário do seu setor. Nesta fase não se escreve o sistema: escreve-se o dado. O que se exige é medição correta, critério declarado e registro rastreável. A fase termina com o inventário completo na mão.

Fase 2 — Sistema (15/10 a 12/11). Com o dado pronto, começa o software. Cada conteúdo novo da disciplina entra direto no projeto na semana em que é dado: buscas em 21 e 22/10, ordenações de 28/10 a 04/11.

A ordem é deliberada. Um sistema construído antes de existir dado é um sistema construído no vácuo, e um inventário feito na última semana é um inventário inventado. Separando as duas coisas, cada uma tem o tempo de que precisa — e a coleta, que é a etapa mais sujeita a atraso por causa de acesso, agenda e clima, sai da frente antes de a implementação começar.

5. Regras do jogo: escopo técnico

O projeto usa exatamente o que a disciplina ensinou, e nada além disso.

Permitido e esperado

- Python 3, executado no terminal (`python main.py`), com **menu numerado**.

- Apenas a **biblioteca padrão** do Python. Os módulos `json`, `pickle`, `os` e `time` bastam.
- Variáveis, operadores, entrada e saída; decisão simples, composta, aninhada e múltipla escolha.
- Laços `for` e `while`, laços aninhados, sentinela.
- Listas, listas de listas (matrizes), dicionários, listas de dicionários, dicionários aninhados.
- Funções com parâmetros, retorno e parâmetros padrão; organização em módulos importados.
- Arquivos de texto: `open`, `with`, leitura e gravação; `try/except` para entrada inválida.
- Persistência com `json` (recomendado) ou `pickle`.
- Recursividade.
- Busca sequencial, busca binária iterativa e busca binária recursiva.
- Ordenação por Bubble Sort, Selection Sort e Insertion Sort, **implementados pelo grupo**.

Não permitido

- Qualquer biblioteca externa: `pandas`, `numpy`, `matplotlib`, `tkinter`, `flask`, `django`, `rich` e assemelhados. Nenhum `pip install`.
- Interface gráfica, página web, aplicativo. O programa roda no terminal.
- Classes e orientação a objetos — não é conteúdo desta disciplina.
- Banco de dados. A persistência é em arquivo.
- `sorted()`, `list.sort()`, `min()`, `max()` e `in` **no lugar** dos algoritmos que o grupo tem de implementar. O ranking e as buscas do relatório precisam sair do código do grupo. O `sorted()` é permitido **apenas como verificação**, em uma função separada e claramente identificada, para conferir se a sua ordenação chegou ao mesmo resultado (ver seção 7.3).

Uso de recurso fora do escopo zera o critério correspondente. O limite não é burocracia: um projeto de 2º período que usa `pandas` deixa de exercitar justamente aquilo que a disciplina precisa avaliar.

6. A estrutura de dados

6.1 O registro de barreira

Cada barreira encontrada é um **dicionário** com estes campos, todos obrigatórios:

```
{
    "codigo": "BIB-TER-04",          # SETOR-SUBSETOR-NN, unico no inventario
    "ambiente": "Sala de Leitura Coletiva",
    "tipo": "porta_estreita",        # categoria da barreira
    "descricao": "Porta de acesso com 72 cm de vao livre",
    "medida": 72.0,                  # valor medido em campo
    "unidade": "cm",
    "referencia_norma": 80.0,        # valor de referencia (Anexo C)
    "gravidade": 4,                  # 1 a 5, atribuido pelo grupo com critério escrito
    "pessoas_dia": 180,              # alcance: pessoas afetadas por dia
    "custo": 1650.00,                # custo estimado da intervencao, em reais
    "intervencao": "Substituicao do marco e da folha da porta",
    "coletado_por": "Ana",
    "data_coleta": "2026-10-20"
}
```

Três desses campos **não são medição, são decisão do grupo**: `gravidade`, `pessoas_dia` e `custo`. Cada um precisa de um critério escrito no relatório — como a escala de gravidade foi definida, como o alcance foi estimado, de onde vem o custo. É nessas três decisões que os grupos vão se diferenciar, e é sobre elas que a arguição vai incidir.

6.2 A árvore do setor

O setor **não** é uma lista de barreiras. É uma hierarquia, porque o campus é uma hierarquia: um setor tem subsetores, que têm ambientes, que podem ter subambientes. A profundidade varia de setor para setor e de ramo para ramo.

```
setor = {
    "nome": "Bloco B - Biblioteca",
    "barreiras": [ ... ],            # barreiras do proprio nivel
    "subsetores": [                  # cada subsetor tem a mesma forma
        {"nome": "Pavimento Terreo",
         "barreiras": [ ... ],
         "subsetores": [
             {"nome": "Sala de Leitura",
              "barreiras": [ ... ],
              "subsetores": [ ... ]}
         ]}
    ]
}
```

Preste atenção nesta consequência, porque ela define metade do projeto: **como a profundidade varia, nenhuma quantidade fixa de laços aninhados consegue totalizar o setor.** Três laços funcionam num setor de três níveis e falham calado num de quatro. Totalizar uma estrutura assim exige percurso recursivo. Não é uma recursão colocada ali para cumprir conteúdo de plano de ensino: é a única forma correta de fazer a conta.

7. Os módulos e as funções obrigatórias

O programa é dividido em seis arquivos. Cada módulo é importado pelos demais; nenhum deles pede dados ao usuário, exceto o `main.py`.

```
main.py          menu e fluxo do programa
coleta.py        cadastro, validacao, importacao da ficha e persistencia
recursao.py      percurso da hierarquia e escolha do pacote de intervencoes
ordenacao.py     os tres algoritmos de ordenacao e o indice de prioridade
busca.py         as tres buscas
relatorio.py     matriz, tabelas e gravacao do relatorio em arquivo
```

7.1 `coleta.py`

Função	O que faz
`validar_registro(barreira, codigos_usados=None)`	devolve a **lista de problemas** do registro; lista vazia significa válido. Verifica campo ausente, código repetido, gravidade fora de 1..5, custo ≤ 0, medida negativa, descrição curta demais para ser verificável
`validar_setor(no, codigos=None)`	valida **recursivamente** todas as barreiras da árvore
`importar_ficha_campo(caminho)`	lê a ficha de campo digitada pelo grupo (um registro por linha, campos separados por `;`) e devolve `(registros, erros)`
`gravar_inventario(caminho, setor)` / `ler_inventario(caminho)`	persistência em arquivo

7.2 `recursao.py`

Função	O que faz
`custo_total(no)`	soma recursiva do custo do nó e de todos os seus subsetores
`contar_barreiras(no)`	contagem recursiva
`contar_por_tipo(no, tipo)`	contagem recursiva filtrada por tipo
`profundidade(no)`	profundidade da árvore; um nó sem subsetores tem profundidade 1
`achatar(no, caminho="")`	devolve a **lista plana** de todas as barreiras, cada uma com o campo `caminho` preenchido. É a ponte entre a árvore e os módulos de ordenação e busca
`caminho_da_barreira(no, codigo)`	devolve `"Bloco B > Térreo > Sala de Leitura"` ou `None`
`resumo_hierarquico(no, nivel=0)`	imprime a árvore com recuo, totalizando cada nível
`melhor_pacote_exaustivo(barreiras, orcamento, pesos, indice=0, contador=None)`	ver 7.5

Toda função recursiva deste módulo precisa ter **caso base identificado em comentário** no código.

7.3 `ordenacao.py`

Função	O que faz
`indice_prioridade(barreira, pesos)`	o índice de prioridade do grupo (seção 7.4)
`calcular_indices(lista, pesos)`	grava o índice dentro de cada registro, **uma vez**, antes de ordenar
`bubble_sort(lista, chave="indice", crescente=False)`	devolve `(lista_ordenada, comparacoes, trocas)`
`selection_sort(lista, chave="indice", crescente=False)`	idem
`insertion_sort(lista, chave="indice", crescente=False)`	idem
`confere_com_o_python(ordenada, chave, crescente)`	compara o resultado próprio com `sorted()` e devolve `True` / `False`

Regras:

- As três funções recebem a **chave** e o **sentido** como parâmetros com valor padrão. A mesma função tem de servir para ordenar por índice (decrescente) e por código (crescente).
- As três **não alteram a lista recebida**: trabalham sobre uma cópia.
- Os contadores `comparacoes` e `trocas` são obrigatórios. Sem eles não há achado A1, e sem o achado A1 não há nota no critério C4.
- `confere_com_o_python` é o único lugar do projeto onde `sorted()` pode aparecer.

7.4 O índice de prioridade — a decisão do grupo

O índice é o número que ordena as barreiras. A **forma** é dada:

```
beneficio = fator_1 * peso_1 + fator_2 * peso_2 + ...
indice    = beneficio / (custo / 1000)        # beneficio por mil reais investidos
```

O que o grupo decide:

1. **Quais fatores** entram no benefício. Obrigatoriamente `gravidade` e `pessoas_dia`; o grupo pode acrescentar **um** terceiro fator de sua autoria, desde que ele venha de um campo coletado.
2. **Quais pesos**, e por quê.
3. **A regra de desempate.** Empates acontecem de verdade — no setor de referência usado para calibrar este projeto, duas barreiras têm índice idêntico (12,857). Sem regra de desempate declarada, a ordem entre elas depende do algoritmo usado, e o relatório fica indefensável.

As três decisões vão escritas no relatório, com justificativa. Um grupo que copia os pesos de outra fonte e não sabe defendê-los perde no critério C5 e na arguição.

7.5 `busca.py`

Função	Devolve
`busca_sequencial(lista, codigo)`	`(posicao, comparacoes)`; `-1` quando não encontra
`busca_binaria_iterativa(lista, codigo)`	idem; exige lista ordenada por código
`busca_binaria_recursiva(lista, codigo, inicio=0, fim=None, contador=None)`	idem, em versão recursiva, com os dois casos base identificados em comentário

7.6 O pacote de intervenções

Duas estratégias, obrigatoriamente as duas:

Estratégia gulosa. Ordene pelo índice de prioridade e vá incluindo barreiras enquanto o dinheiro couber. Rápida e intuitiva.

Estratégia exaustiva. `melhor_pacote_exaustivo` percorre recursivamente todas as combinações possíveis — para cada barreira, o ramo que a inclui e o ramo que não a inclui — e devolve a combinação de **maior benefício total** que respeita o orçamento, junto com o número de chamadas recursivas realizadas.

```
def melhor_pacote_exaustivo(barreiras, orcamento, pesos, indice=0, contador=None):
    # caso base: nao ha mais barreiras a decidir
    # ramo 1: nao incluir barreiras[indice]
    # ramo 2: incluir barreiras[indice], se couber no orcamento
    # devolve (beneficio, custo, lista_de_codigos, chamadas)
    ...
```

Antes de chamar a exaustiva, descarte as barreiras que **sozinhas** já estouram o orçamento: é uma poda barata e o grupo tem de explicar por que ela é legítima. O programa deve **avisar o usuário** quando o número de candidatos passar de 20, e o grupo tem de saber dizer por quê.

7.7 `relatorio.py` e o menu

`relatorio.py` monta a **matriz ambiente × tipo de barreira** (uma lista de listas, com as contagens), imprime as tabelas alinhadas no terminal e grava o relatório em arquivo de texto.

O `main.py` oferece, no mínimo:

```
=== CAMPUS ACESSIVEL - setor: <nome do setor> ===
1. Importar ficha de campo
2. Cadastrar / editar / remover barreira
3. Ver estrutura do setor (resumo hierarquico)
4. Ranking de prioridade  (bubble / selection / insertion)
5. Localizar barreira por codigo  (sequencial / binaria iterativa / binaria recursiva)
6. Matriz ambiente x tipo de barreira
7. Plano de intervencao sob orcamento  (guloso x exaustivo)
8. Relatorio de desempenho dos algoritmos
9. Gravar relatorio em arquivo
0. Sair
```

As opções 4, 5 e 7 **perguntam ao usuário qual algoritmo usar** e exibem os contadores ao final. O programa serve para medir, não só para responder.

8. A coleta de campo

Cada grupo recebe um setor e vai a campo com a ficha do **Anexo A**, uma trena e um celular. A coleta é feita **pelo grupo**, presencialmente, com todos os integrantes participando de pelo menos uma saída.

A coleta tem duas janelas. Da aula de lançamento até **24/09** vai o **piloto**: oito barreiras, para calibrar a ficha, a escala de gravidade e o método de custo antes de levantar o setor inteiro. De 25/09 a **15/10** vai o **levantamento completo**. Trate o piloto como ensaio de método: o que der errado nele — unidade trocada, campo que não serve, gravidade impossível de atribuir — é justamente o que você não quer descobrir na décima quinta barreira.

Regras mínimas:

- **Mínimo de 15 barreiras** no inventário final, distribuídas em pelo menos **3 ambientes** e **2 níveis** de hierarquia.
- Cada barreira tem **medida em campo** com unidade, não estimativa de olho.
- Cada barreira tem **uma foto** identificada pelo código. As fotos vão em anexo ao relatório.
- Nada de fotografar pessoas. Nada de entrar em ambiente em uso sem autorização. Ver o termo de conduta no **Anexo B**.
- A **carta de apresentação da coordenação** deve ser portada durante a coleta.

O custo da intervenção é estimativa do grupo, e estimativa tem método: cotação em loja ou site, orçamento informal com prestador, comparação com serviço equivalente. O método vai escrito no relatório. Custo sem método declarado é tratado como dado não verificável.

9. Os cinco achados — o coração da nota

O relatório técnico se organiza em torno de cinco achados. Um achado tem sempre a mesma forma: **um número medido nos dados do grupo, e a explicação de por que aquele número é aquele.** Número sem explicação não é achado; explicação sem número, muito menos.

A1 — Achado de ordenação

Tabela com **comparações e trocas** dos três algoritmos, em três tamanhos de base (um terço, dois terços e o total das suas barreiras) e em três ordens iniciais: a ordem em que os dados foram coletados, a base já ordenada pelo índice, e a base em ordem inversa.

O grupo explica: por que dois dos três algoritmos fazem sempre o mesmo número de comparações, independentemente da ordem inicial; por que o terceiro varia tanto; e qual é a relação entre o número de trocas do Bubble e do Insertion. Há um padrão exato nesses números, e ele se enuncia em uma linha.

A2 — Achado de busca

Tabela com o número de comparações da busca sequencial, da binária iterativa e da binária recursiva para **cinco códigos**: o primeiro da lista ordenada, o do meio, o último, um qualquer do

meio da lista e **um que não existe**.

O grupo explica: em qual dos cinco casos a busca sequencial ganha da binária e por quê; por que a binária precisa da lista ordenada e quanto custa produzir essa ordenação; e por que o custo da binária para um código inexistente é o mesmo do pior caso de um código existente.

A3 — Achado de hierarquia

Custo total e contagem de barreiras **por subsetor de primeiro nível**, a profundidade real da árvore do seu setor, e o caminho hierárquico completo da barreira mais grave.

O grupo explica: em que ponto exato uma versão com laços aninhados fixos passaria a dar resultado errado no **seu** setor, e o que aconteceria com a função recursiva se o caso base fosse removido — indicando o erro que o Python apresentaria.

A4 — Achado de otimização

Para pelo menos **quatro tetos de orçamento diferentes**, a tabela comparando o pacote guloso e o pacote exaustivo: benefício total, custo total, número de itens e número de chamadas recursivas.

O grupo precisa encontrar **pelo menos um orçamento em que as duas estratégias divergem**, mostrar quais barreiras cada uma escolheu e explicar exatamente por que a gulosa perdeu naquele caso. Existe sempre um: se você não achou, não testou o suficiente.

O grupo também mede o número de chamadas recursivas para vários tamanhos de entrada e enuncia a relação entre o tamanho da entrada e o número de chamadas. Depois responde: se o campus inteiro tem 400 barreiras, essa abordagem serve? Por quê?

A5 — Achado de decisão

O índice de prioridade do grupo, com os fatores, os pesos, a regra de desempate e a justificativa de cada escolha. Em seguida, a **análise de sensibilidade**: refaça o ranking dobrando o peso da gravidade e depois dobrando o peso do alcance, e mostre o que muda nas dez primeiras posições.

O grupo então responde à pergunta que fecha o projeto: **diante dessas variações, o grupo mantém a recomendação original?** Se mantém, com qual argumento; se muda, o que o fez mudar.

10. O caderno de bordo

Um arquivo aberto no primeiro dia e fechado na entrega. Uma entrada por sessão de trabalho, com data, quem participou, o que foi feito, **o que não funcionou** e o que foi decidido. Entradas de duas linhas bastam; entradas inventadas na véspera aparecem.

Três seções obrigatórias ao final:

Divisão de responsabilidades. Quem fez o quê, por módulo e por etapa.

Registro de uso de ferramentas de IA. Ferramentas de IA são instrumento legítimo de trabalho e serão instrumento legítimo na sua profissão. Neste projeto elas são de uso permitido e de **declaração obrigatória**: para cada uso, registre o que foi pedido, o que voltou, e **o que o**

grupo alterou no que voltou, e por quê. O que se avalia aqui não é o quanto o grupo usou; é a qualidade da crítica que o grupo fez ao resultado.

Um defeito que nós encontramos. Descreva um defeito real que apareceu no projeto — de código gerado, de código de colega ou de código próprio —, o teste ou a execução que revelou o defeito, e a correção. Um defeito por grupo, com evidência.

11. Entregas parciais

As datas são as do plano de ensino da disciplina, iguais para as duas turmas.

	Fase	Data	O que entrega	Onde
E1	1	**24/09, quinta**	**Inventário-piloto:** ficha de campo com no mínimo 8 barreiras, digitada em arquivo de texto · desenho da árvore do setor · critério da escala de gravidade e método de estimativa de custo, escritos	AVA, até 23h59
E2	1→2	**15/10, quinta**	**Inventário completo:** no mínimo 15 barreiras, em 3 ambientes e 2 níveis de hierarquia · ficha digitada · uma foto por barreira · caderno de bordo em dia	Em aula + AVA
E3	2	**05/11, quinta**	`coleta.py` e `recursao.py` funcionando · resumo hierárquico imprimindo · ranking de prioridade saindo de pelo menos uma das ordenações próprias	Em aula + AVA
E4	2	**12/11, quinta**	**Pacote final, congelado.** Sistema completo · relatório técnico com os cinco achados · caderno de bordo fechado · relatório `.txt` gerado pelo programa · ficha de revisão por pares assinada pelo grupo revisor	Em aula + AVA, até 23h59
E5	2	**25 ou 26/11**	Pitch de 5 minutos, demonstração ao vivo e arguição	Em aula

E3 e E4 têm defesa oral de 3 minutos por um integrante sorteado na hora, sobre o estado do trabalho. O sorteio é **sem reposição**: quem defendeu em E3 não é sorteado em E4. Consequência prática: em grupos de 4, dois integrantes já terão falado publicamente antes da apresentação final; em grupos de 3, dois dos três.

Na semana da Prova 01 (28/09 a 01/10) nada do projeto vence. A coleta continua correndo, no ritmo do grupo.

O código congela em 12/11. Nada entregue depois dessa data entra na avaliação técnica. Entre 12/11 e a apresentação, o trabalho do grupo é preparar a defesa — e ninguém ganha vantagem por apresentar no segundo bloco.

Calendário completo do projeto

Data	Dia	O que acontece
16/09	**qua**	**Lançamento · formação dos grupos · sorteio dos setores** — Eng. Computação
17/09	**qui**	**Lançamento · formação dos grupos · sorteio dos setores** — Sistemas de Informação
17/09 a 24/09		Coleta do inventário-piloto
24/09	**qui**	**E1 — inventário-piloto, árvore do setor e critérios**
30/09–01/10	qua/qui	Revisão e **Prova 01** · nada do projeto vence nesta semana
07–14/10		Programa espelho em aula · coleta do inventário completo em campo
15/10	**qui**	**E2 — inventário completo · início da Fase 2: o sistema**
21–22/10	qua/qui	Buscas sequencial, binária iterativa e binária recursiva
28/10–04/11	qua/qui/qua	Bubble, Selection e Insertion Sort
05/11	**qui**	**Checkpoint 1 — E3 + defesa oral sorteada**
11/11	qua	Desenvolvimento assistido: persistência e integração dos módulos
12/11	**qui**	**Checkpoint 2 — E4, pacote congelado + revisão por pares + defesa oral sorteada**
18/11	qua	Prova Interdisciplinar — sem aula da disciplina
19/11	**qui**	**Ensaio do pitch e checklist final de apresentação**
25/11	**qua**	**Apresentações — bloco 1**
26/11	**qui**	**Apresentações — bloco 2**
02/12	qua	Grupos remanescentes, painel comparativo e devolutiva coletiva
03/12	qui	Prova 02
17/12	qui	Devolutiva individual de notas

12. A apresentação final

Formato: pitch de **5 minutos**, cronometrado, com corte seco em 5:00. Em seguida, **4 minutos de arguição**.

Os dois sorteios. Com o grupo já à frente da turma:

1. Sorteia-se **quem apresenta**. Essa pessoa faz o pitch inteiro sozinha. Ninguém do grupo pode complementar, corrigir ou sinalizar.
2. Sorteia-se **quem responde** à arguição, entre os demais integrantes. Igualmente sozinha.

Os dois sorteios são feitos na hora, na frente da turma, entre **todos** os integrantes presentes. Não há como dividir o projeto em fatias e cada um decorar a sua: qualquer integrante pode cair em qualquer um dos dois papéis.

Estrutura obrigatória do pitch — 5 minutos

Tempo	Bloco
0:00–0:30	O setor e o problema: o que vocês foram olhar e o que encontraram
0:30–1:30	O que medimos: quantas barreiras, em quantos ambientes, quanto custaria resolver tudo
1:30–3:00	A recomendação: o teto de orçamento adotado, o pacote escolhido, e **por que esse e não outro**
3:00–4:30	**O achado que mais nos surpreendeu** — um dos cinco, com o número na tela
4:30–5:00	O que faríamos diferente se recomeçássemos

Recursos: no máximo **5 slides**, e pelo menos um deles tem de ser uma **tabela de números do próprio grupo**. A demonstração ao vivo do programa é obrigatória e conta dentro dos 5 minutos — deixe o terminal aberto e o arquivo carregado antes de começar.

A arguição. São perguntas sobre o que o grupo entregou: o código do grupo, os números do grupo, as decisões do grupo. Uma delas é sempre desta forma: o professor propõe uma alteração pequena e concreta — trocar um peso, inverter a lista de entrada, remover uma linha — e **quem foi sorteado diz, antes de executar, o que vai acontecer.** Depois se executa e se compara. Prever corretamente o comportamento do próprio programa é a evidência mais direta de que o grupo entende o que construiu.

13. Como a nota é composta

Nota do PJ de 0,0 a 10,0, distribuída em sete critérios. Cada critério é avaliado em **cinco níveis ordenados (0 a 4)** e convertido em pontos por `nível ÷ 4 × pontos do critério`.

	Critério	Pontos
C1	**Dados** — rastreabilidade, cobertura e qualidade da coleta de campo	1,0
C2	**Sistema** — corretude, aderência ao escopo, organização em módulos	1,0
C3	**Recursão** — necessidade, caso base, percurso da hierarquia e pacote exaustivo	1,5
C4	**Ordenação e busca** — implementação própria e medição com contadores	1,0
C5	**Os cinco achados** — qualidade da medição e da explicação	1,0
C6a	**Pitch** — clareza, estrutura, uso dos 5 minutos, demonstração ao vivo	1,0
C6b	**Arguição e previsão** — sustentação do trabalho pelo integrante sorteado	2,0
C7	**Processo** — entregas parciais no prazo e completas, caderno de bordo, revisão por pares	1,5
	Total	**10,0**

A nota é do grupo, com uma exceção. Um integrante que, ao ser sorteado — em qualquer um dos dois checkpoints ou na arguição final —, não conseguir sustentar o trabalho que o grupo assinou, recebe **ajuste individual de até −2,0** sobre a nota do PJ, registrado em ficha e comunicado na devolutiva. Não há ajuste positivo: ninguém sobe sozinho.

14. Integridade acadêmica

- **Dado de campo não verificável** (medida que não se confirma em conferência, foto ausente, custo sem método) zera o critério C1 e obriga o grupo a refazer a coleta daquela barreira.

- **Recurso fora do escopo** zera o critério correspondente.
- **`sorted()` ou `.sort()` no lugar da ordenação própria** zera o critério C4.
- **Uso de IA não declarado**, quando houver evidência, zera o critério C7 e é registrado como ocorrência acadêmica. Uso declarado, qualquer que seja o volume, **não penaliza**.
- **Código que o grupo não explica** não é código do grupo para fins de avaliação. Esta é a regra que resume todas as outras.

15. Checklist de entrega — 12/11

```
[ ] main.py, coleta.py, recursao.py, ordenacao.py, busca.py, relatorio.py
[ ] inventario.json com no minimo 15 barreiras, 3 ambientes, 2 niveis
[ ] ficha_de_campo.pdf preenchida e assinada pelo grupo
[ ] fotos/ com uma foto por barreira, nomeada pelo codigo
[ ] relatorio.pdf de 4 a 6 paginas com os achados A1 a A5
[ ] relatorio.txt gerado pelo proprio programa
[ ] caderno_de_bordo.pdf com as tres secoes obrigatorias
[ ] ficha_revisao_por_pares.pdf assinada pelo grupo revisor
[ ] o programa roda com "python main.py" em uma maquina que nao e a de voces
[ ] todas as funcoes recursivas tem o caso base identificado em comentario
[ ] todo numero citado no relatorio pode ser reproduzido rodando o programa
```

O penúltimo item é literal: teste em outra máquina antes de entregar. O último também: em cada número do relatório, o professor pode pedir "mostre onde isso sai".

Anexo A — Ficha de campo

Uma linha por barreira. Preencha a lápis em campo e digite depois no formato aceito por `importar_ficha_campo`, com os campos separados por ponto e vírgula.

Campo	Como preencher
`caminho`	níveis separados por `>`, ex.: `Bloco B > Térreo > Hall`
`codigo`	`SETOR-SUBSETOR-NN`, sequencial e único
`ambiente`	nome do ambiente como ele é conhecido no campus
`tipo`	uma das categorias da lista abaixo
`descricao`	o que foi observado, com o valor medido dentro da frase
`medida` / `unidade`	valor medido em campo e a unidade
`referencia_norma`	valor de referência do Anexo C
`gravidade`	1 a 5, conforme a escala definida pelo grupo
`pessoas_dia`	alcance estimado, com método declarado
`custo`	estimativa em reais, com método declarado
`intervencao`	o que precisa ser feito, em uma frase
`coletado_por`	nome de quem mediu
`data_coleta`	`AAAA-MM-DD`

Categorias de tipo — use estas; se precisar de outra, registre no caderno de bordo.

`degrau_isolado` · `desnivel_soleira` · `piso_irregular` · `rampa_inclinada` · `ausencia_corrimao` · `porta_estreita` · `circulacao_estreita` · `balcao_alto` · `interruptor_alto` · `mobiliario_inadequado` · `banheiro_inadequado` · `sinalizacao_ausente` · `vaga_ausente` · `elevador_ausente`

Modelo de linha digitada

```
caminho;codigo;ambiente;tipo;descricao;medida;unidade;referencia_norma;gravidade;pessoas_dia;custo;intervencao;coletado_por;data_coleta
Bloco B > Terreo > Hall;BIB-TER-03;Balcao de atendimento;balcao_alto;Balcao com 1,10 m de altura em toda a extensao;110;cm;90;4;210;2400,00;Rebaixamento de trecho de 90 cm;Bruno;2026-10-20
```

Anexo B — Termo de conduta na coleta

1. Portar a carta de apresentação da coordenação durante toda a coleta e apresentá-la quando solicitado.
2. Não fotografar pessoas. As fotos registram o ambiente e a barreira.
3. Não entrar em ambiente em uso — aula, atendimento, culto, reunião — sem autorização de quem está responsável.
4. Não medir nada que exija subir, escalar, mover móvel pesado ou desligar equipamento.
5. Não abordar funcionários para pedir dados ou opinião: a coleta é observacional. Dúvidas sobre o ambiente vão ao professor.
6. Registrar no caderno de bordo qualquer recusa de acesso, em vez de insistir.
7. Não divulgar os achados fora do projeto antes da devolutiva. O relatório é documento de trabalho da coordenação.

Anexo C — Valores de referência para a coleta

Estes valores orientam o preenchimento do campo `referencia_norma`. **São referência de partida, não citação normativa:** antes de usá-los no relatório, o grupo tem de conferi-los na NBR 9050 vigente e citar a seção consultada. Divergência entre o valor daqui e o da norma é achado, não erro — registre.

Item	Valor de referência
Vão livre mínimo de porta	80 cm
Largura mínima de circulação para uma pessoa	90 cm
Inclinação máxima de rampa (desnível até 1,50 m)	8,33% (1:12)
Desnível que não requer tratamento	até 5 mm
Desnível entre 5 mm e 2 cm	rampa, inclinação máxima 1:2
Desnível acima de 2 cm	tratado como degrau
Altura de balcão de atendimento acessível	até 90 cm, com altura livre inferior de 73 cm
Faixa de alcance para comandos e interruptores	60 cm a 100 cm do piso
Corrimão em rampa ou escada	duas alturas, 70 cm e 92 cm, nos dois lados
Vagas reservadas em estacionamento	2% do total, no mínimo uma
Módulo de referência (cadeira de rodas)	0,80 m × 1,20 m
Área para rotação de 360°	círculo de 1,50 m de diâmetro